

AREA SCIENCE PARK
RESEARCH AND TECHNOLOGY INSTITUTE
LABORATORY OF DATA ENGINEERING

Web Application for DPCfam and DPCstruct Data Exploration

Report 10 (17 March - 07 April 2026) : Web Development
(DPCStruct Backend)

Emmanuel Nyandu Kagarabi
Master in Data Management and Curation (MDMC - SISSA)

Supervisors :
Valerio Piomponi
Elaheh Saadat

April 8, 2026

1 Introduction

This report is the tenth in a sequence documenting the weekly assigned tasks of our internship. The project has been updated in such a way that the Metaclusters/DPCFam page displays by default the paginated table of all DPCFam metaclusters; however, if desired, users can filter for DPCFam Standard (≥ 50 seeds/MCID) or DPCFamB ($25 \leq \text{seeds/MCID} < 49$), summing up to three possible choices (**DPCFam**, **DPCFam Standard**, and **DPCFamB**). Regarding DPCstruct, users can now explore the 3D structures of representative PDBs (cluster center or domain with the highest pLDDT) per MCID, which was made possible by initializing the Mol* toolkit in the user’s browser.

2 Development

2.1 DPCfam Update

In Report 9, we updated the DPCFam app by merging metaclusters from both **DPCFam Standard** and **DPCFamB**:

Table 1: DPCFam in Numbers.

Set	Total Size (Number of MCs)	Size UniRef50		Average Length (aa)	
		Min	Max	Min	Max
DPCFam Standard	46,828	50	131,827	50	3,150
DPCFamB	34,556	25	49	50	3,211
DPCFam	81,384	25	131,827	50	3,211

As stated in (Russo et al., 2022) and summarized in Table 1, within a given metacluster, each domain (seed sequence) has at least 50 amino acids. Based on the number of domain sequences per metacluster, we have a “rich” set (**DPCFam Standard**: at least 50 elements per MCID) and **DPCFamB** (where each MCID has at least 25 domains but less than 50, often considered lower quality). Using the size/cardinality of each metacluster, we have implemented a mechanism allowing users to select **DPCFam Standard**, **DPCFamB**, or their union (**DPCFam**, which is the default display combining the two disjoint subsets).

2.2 DPCstruct Update

For the DPCstruct app, our main focus over the past two weeks was to implement a mechanism allowing users to internally explore the 3D structures of representative PDBs per metacluster. We agreed to use the Mol* toolkit, which is, currently, the optimal tool for this type of task.

2.2.1 Mol*: Overview

Mol* is a modern, open-source JavaScript library for the 3D visualization of macromolecular structures in the web browser. It was jointly developed by **PDBe** (Protein Data Bank in Europe, EMBL-EBI) and **RCSB PDB** (Research Collaboratory for Structural Bioinformatics), the two largest protein structure databases in the world, and serves as their primary 3D viewer. It is:

- **Client-side:** It runs entirely in the browser using **WebGL** (Web Graphics Library), the same technology that powers browser-based 3D games. This means no server-side GPU is needed. WebGL provides a JavaScript API to communicate directly with the user’s Graphics Processing Unit (GPU).
- **Feature-rich:** It allows users to rotate, zoom, measure distances, color by property, take screenshots, switch representations, and view the amino acid sequence.
- **Open-source:** It is released under the MIT license and is actively maintained.
- **Embeddable:** It can be loaded from a CDN using only two files (one CSS and one JS) and initialized with a few lines of JavaScript. A Content Delivery Network (CDN) is a system of distributed servers that cache and deliver files quickly based on the user’s location.

2.2.2 PDB Format: Overview

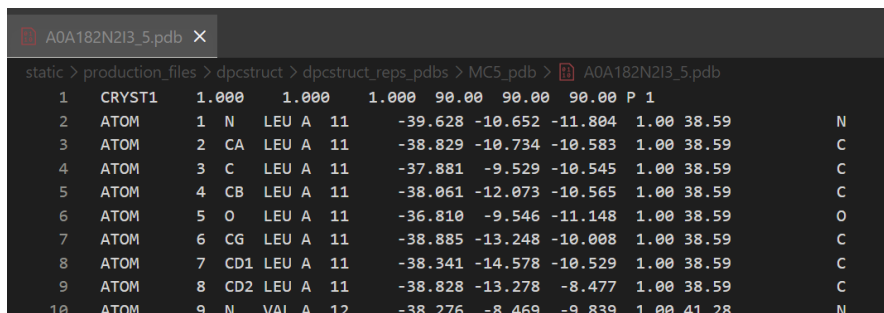
A **.pdb** file (**Protein Data Bank** format) is a plain text file that stores the 3D coordinates of every atom in a protein structure. Created in 1971, it remains the most widely used format for representing molecular structures.

The PDB format is *column-based* and *fixed-width*, meaning every data field occupies a specific range of columns. Each line is at most 80 characters wide and begins with a 6-character **record type** keyword (left-justified and padded with spaces), as shown in Table 2:

Table 2: Major Record Categories (e.g : **2HBS** (1997))

Record	Section	Meaning
HEADER	Title	Molecule class, deposition date, PDB ID
TITLE	Title	Free-text description of the structure
SEQRES	Primary structure	Amino acid sequence
CRYST1	Crystallographic	Unit cell dimensions and space group
ATOM	Coordinates	Standard amino acid atom coordinates
HETATM	Coordinates	Ligands, waters, modified residues
TER	Coordinates	End of a polymer chain
CONECT	Connectivity	Explicit bonds (mainly for HETATM)
END	Bookkeeping	Last record in file
MODEL/ENDMDL	Coordinates	Bracket one conformer in multi-model files

A typical representative PDB (Figure 1 - AlphaFold2 prediction) contains some records among those listed above :



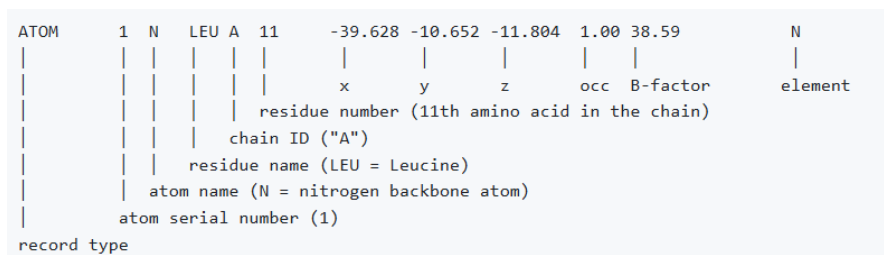
Record Number	Record Name	Field 1	Field 2	Field 3	Field 4	Field 5	Field 6	Field 7	Field 8	Field 9	Field 10	Field 11
1	CRYST1	1.000	1.000	1.000	90.00	90.00	90.00	P	1			
2	ATOM	1	N	LEU	A	11	-39.628	-10.652	-11.804	1.00	38.59	N
3	ATOM	2	CA	LEU	A	11	-38.829	-10.734	-10.583	1.00	38.59	C
4	ATOM	3	C	LEU	A	11	-37.881	-9.529	-10.545	1.00	38.59	C
5	ATOM	4	CB	LEU	A	11	-38.061	-12.073	-10.565	1.00	38.59	C
6	ATOM	5	O	LEU	A	11	-36.810	-9.546	-11.148	1.00	38.59	O
7	ATOM	6	CG	LEU	A	11	-38.885	-13.248	-10.008	1.00	38.59	C
8	ATOM	7	CD1	LEU	A	11	-38.341	-14.578	-10.529	1.00	38.59	C
9	ATOM	8	CD2	LEU	A	11	-38.828	-13.278	-8.477	1.00	38.59	C
10	ATOM	9	N	VAL	A	12	-38.276	-8.469	-9.839	1.00	41.28	N

Figure 1: MC5 Representative : CRYST1- ATOM -TER Records.

Let's elaborate a bit on the main records available Figure 1:

The first line (CRYST1) is a dummy crystallographic cell. AlphaFold structures are not crystals; this line exists only because the PDB format requires it. Values 1.000 1.000 1.000 mean the unit cell axes have length 1 angstrom ($1\text{\AA} = 0.1\text{nm}$)-a placeholder. **Mol*** ignores this record for predicted structures.

Every ATOM line follows a strict column-based format. Let's decode one:



ATOM	1	N	LEU	A	11	-39.628	-10.652	-11.804	1.00	38.59	N
						x	y	z	occ	B-factor	element
						residue number (11th amino acid in the chain)					
						chain ID ("A")					
						residue name (LEU = Leucine)					
						atom name (N = nitrogen backbone atom)					
						atom serial number (1)					
	record type										

Figure 2: ATOM Record.

In general, for ATOM records, columns are interpreted as :

- Record Name (ATOM): Indicates this line describes a standard atom.
- Atom Serial Number (1, 2): A unique ID for the atom.
- Atom Name (N, CA, C,...): The standard backbone are presented in Table 3:

Table 3: Atoms.

Atom name	Element	Role
N	N	Backbone nitrogen (peptide bond)
CA	C	Alpha-carbon (chiral centre of residue)
C	C	Backbone carbonyl carbon
O	O	Backbone carbonyl oxygen
CB	C	Beta-carbon (first side-chain carbon)

- Residue Name (LEU,MET,...): **Three-letter codes** for residue/amino-acid names. The standard 20 amino acids are presented below in Table 4:

Table 4: Standard Amino-Acids.

Code	1-Ltr	Name	Code	1-Ltr	Name
ALA	A	Alanine	LEU	L	Leucine
ARG	R	Arginine	LYS	K	Lysine
ASN	N	Asparagine	MET	M	Methionine
ASP	D	Aspartate	PHE	F	Phenylalanine
CYS	C	Cysteine	PRO	P	Proline
GLN	Q	Glutamine	SER	S	Serine
GLU	E	Glutamate	THR	T	Threonine
GLY	G	Glycine	TRP	W	Tryptophan
HIS	H	Histidine	TYR	Y	Tyrosine
ILE	I	Isoleucine	VAL	V	Valine

- Chain ID (A) & Residue Number (1,...): Identifies which protein chain and sequence position this atom belongs to.
- Orthogonal Coordinates (X, Y, Z): $-39.628 \ -10.652 \ -11.804$ - The exact 3D position in Ångströms ($1\text{\AA} = 10^{-10}m$).
- Occupancy $\in [0, 1]$: the fraction of molecules in the crystal where this atom is at this position. 1.00 (always for predicted structures) means every molecule agrees.
- B-factor : In experimental X-ray crystallography, the B-factor (also called temperature factor) measures how much each atom vibrates. AlphaFold2 repurposes this column to store the **pLDDT score** a per-residue confidence measure. **Mol*** is aware of this convention and can colour the structure by pLDDT when it detects an AlphaFold model.

Table 5: Color Legend

pLDDT range	Interpretation	Color in Mol*
> 90	Very high confidence	Dark Blue
70 – 90	Confident	Light Blue or Cyan
50 – 70	Low confidence	Yellow
< 50	Very low confidence	Orange

- Element symbol(N) : Chemical element (N = Nitrogen)... Cfr Table 3.

TER record marks the end of chain A:

1057	ATOM	1056	OD1	ASN A 149	-5.834	3.589	48.391	1.00	92.12	0
1058	TER	1057		ASN A 149						

This tells parsers that chain A ends at Asyne 149. It is mandatory in PDB format. Mol* reads it to correctly identify chain boundaries.

In mathematical terms: each ATOM line gives us a point in $\mathbb{R}^3$ (the x, y, z coordinates) labeled with metadata about which atom and amino acid it represents. The entire PDB file is thus a labeled point cloud in 3D space.

2.3 Implementation of Mol* in our Project

Recently, we wrote the `dpcstruct_reps_pdb.ipynb` notebook, which organizes representative PDBs into per-MC subfolders. Building on that, we summarize our implementation in six main-steps:

Step 1 : Django (The Backend) - views.py

```

1084 class DpcstructDetailView(DetailView):
1085     def get_context_data(self, **kwargs):
1086         """
1087         # To return list of .pdb files in this MC's folder
1088         pdb_folder = os.path.join(
1089             settings.BASE_DIR, 'static',
1090             'production_files', 'dpcstruct', 'dpcstruct_reps_pdb',
1091             f'{mc_id}.pdb'
1092         )
1093         pdb_files = []
1094         if os.path.isdir(pdb_folder):
1095             for fname in sorted(os.listdir(pdb_folder)):
1096                 if fname.endswith('.pdb'):
1097                     pdb_files.append({
1098                         'name': fname,
1099                         'url': static(
1100                             f'production_files/dpcstruct/dpcstruct_reps_pdb/'
1101                             f'{mc_id}.pdb/{fname}'
1102                         ),
1103                     })
1104         context['pdb_files'] = pdb_files
1105
1106         # Split Pfam labels if valid
1107         if self.object.pfam_da and self.object.pfam_da != 'UNKNOWN':
1108             context['pfam_label_list'] = [l.strip() for l in self.object.pfam_da.split('-')]
1109         else:
1110             context['pfam_label_list'] = []
1111
1112         return context

```

Figure 3: Python reads our folders, finds the relevant .pdb file locations, and sends this data (names, urls/paths) to our HTML template (`dpcstruct_detail.html`) via a context variable called `pdb_files`.

Step 2 : Loading the Mol* Library

Mol* requires two things: Styles (CSS) and Code (JavaScript). We load these using a CDN : [jsdelivr.net](https://cdn.jsdelivr.net) is a free CDN popular for open-source JavaScript libraries.

At the top of the file, we include the CSS:

```
7 <!-- Mol* Viewer CSS -->
8 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/molstar@latest/build/viewer/molstar.css">
```

At the bottom of the file, we load the JS functionality:

```
209 <!-- Mol* Viewer JS -->
210 <script src="https://cdn.jsdelivr.net/npm/molstar@latest/build/viewer/molstar.js"></script>
```

Step 3 : Preparing the HTML Container

Mol* needs a place to render the 3D graphics. We provide a simple `<div>` (container) and give it a unique id.

```
172 <!-- Mol* container -->
173 <div id="molstar-viewer"></div>
```

To ensure the viewer actually takes up space on the screen, we give it explicit dimensions in our CSS `<style>` block:

```
20 #molstar-viewer {
21   width: 100%;
22   height: 600px;
23   position: relative;
24   border: 1px solid #dee2e6;
25   border-radius: 0.5rem;
26 }
```

Step 4 : Passing Data from Django to JavaScript

Our Django view passes a variable named `pdb_files` (a list of dictionaries, each having a `name` and a `url`). But JavaScript cannot read Python directly. To securely transfer this data from Django to Javascript, we use Django's `json_script` filter:

```
175 <!-- Pass PDB URLs to JavaScript -->
176 <script>{{ pdb_files|json_script:"pdb-files-data" }}</script>
```

It takes the Python list, converts it to JSON (JavaScript Object Notation), and safely embeds it in the HTML as a script block with the ID `pdb-files-data`. Later, JS can extract it.

Step 5 : Building the User Interface Controls

Each Metacluster has at most two PDB files. We build buttons using a `for` loop so the user can switch between them:

```
159 <!-- PDB file selector (only shown if 2+ files) -->
160 {% if pdb_files|length > 1 %}
161 <div class="btn-group mb-3" role="group" aria-label="Select PDB">
162   {% for pdb in pdb_files %}
163     <button type="button"
164       class="btn btn-outline-primary pdb-selector {% if forloop.first %}active{% endif %}"
165       data-pdb-url="{{ pdb.url }}">
166       {{ pdb.name|cut:".pdb" }}
167     </button>
168   {% endfor %}
169 </div>
170 {% endif %}
```

Step 6 : The JavaScript Logic (The Core)

This is where the magic happens. We write this inside `<script>` tags at the bottom.

1. Wait for the browser : Document Object Model (DOM)

```
212 <script>
213 document.addEventListener("DOMContentLoaded", function() {
214     // Activate tab based on URL hash
215     var hash = window.location.hash;
216     if (hash) {
217         var triggerEl = document.querySelector('button[data-bs-target="' + hash + '"]');
218         if (triggerEl) {
219             var tab = new bootstrap.Tab(triggerEl);
220             tab.show();
221         }
222     }
223
224     // Update URL hash when tab is changed
225     var tabList = [].slice.call(document.querySelectorAll('button[data-bs-toggle="tab"]'));
226     tabList.forEach(function (tabEl) {
227         tabEl.addEventListener('shown.bs.tab', function (event) {
228             var targetId = event.target.getAttribute('data-bs-target');
229             history.replaceState(null, null, targetId);
230         });
231     });
232 })
```

This tells JS: “Do not execute anything until the HTML (the DOM) is fully loaded”.

2. Read the Django Data

```
233 // --- Mol* 3D Viewer ---
234 var dataEl = document.getElementById('pdb-files-data');
235 if (!dataEl) return;
236 var PDB_FILES = JSON.parse(dataEl.textContent);
237 if (!PDB_FILES || PDB_FILES.length === 0) return;
```

Here, JS looks for the JSON data we injected in **Step 4**. `JSON.parse` translates it into a usable JavaScript array.

3. Lazy Loading the Viewer

Because 3D rendering is computationally heavy, we don’t want to load it unless the user actually opens the “3D Viewer” tab.

```
239 var viewerInstance = null;
240 var viewerInitialized = false;
241
242 // Initialize Mol* when the 3D Viewer tab is first shown
243 var viewerTab = document.getElementById('viewer-tab');
244 viewerTab.addEventListener('shown.bs.tab', function () {
245     if (!viewerInitialized) {
246         viewerInitialized = true;
247         initView(PDB_FILES[0].url);
248     }
249 });
```

We listen for the `shown.bs.tab` event (a Bootstrap feature). When that tab is clicked, if we haven’t initialized Mol* yet, we call `initViewer()` and pass the first PDB URL in our list.

4. The Initialization Function

```
261 async function initView(pdbUrl) {
262     viewerInstance = await molstar.Viewer.create('molstar-viewer', {
263         layoutIsExpanded: false,
264         layoutShowControls: true,
265         layoutShowRemoteState: false,
266         layoutShowSequence: true,
267         layoutShowLog: false,
268         layoutShowLeftPanel: false,
269         viewportShowExpand: true,
270         viewportShowSelectionMode: true,
271         viewportShowAnimation: false,
272     });
273     await viewerInstance.loadStructureFromUrl(pdbUrl, 'pdb', false);
274 }
```

(a) Initialisation Parameters.

Option	Value	What it does
layoutIsExpanded	false	Don't start in fullscreen mode
layoutShowControls	true	Show the control panel on the right
layoutShowRemoteState	false	Hide the "remote state" button (for sharing sessions — we don't need it)
layoutShowSequence	true	Show the amino acid sequence bar at the top of the viewer
layoutShowLog	false	Hide the debug log panel
layoutShowLeftPanel	true	Show the left panel (structure tree, components)
viewportShowExpand	true	Show the "expand to fullscreen" button
viewportShowSelectionMode	true	Allow users to select atoms/residues
viewportShowAnimation	false	Don't show animation controls

(b) Legend.

- `async / await`: Downloading the PDB file and initializing graphics takes time. `await` pauses the function until Mol* finishes setting up, so the browser doesn't freeze.
- `Viewer.create`: Tells Mol* to target our `molstar-viewer` HTML element and configure the side menus/layout.
- `loadStructureFromUrl`: Actually fetches the `.pdb` data from our server URL and renders it.

5. Handling Button Clicks (Switching Structures)

If a user clicks one of our buttons from **Step 5**, we must update the viewer:

```

276 // Handle PDB selector buttons
277 document.querySelectorAll('.pdb-selector').forEach(function (btn) {
278   btn.addEventListener('click', async function () {
279     // 1. Remove 'active' color from all buttons, add to the clicked one
280     document.querySelectorAll('.pdb-selector').forEach(function (b) {
281       b.classList.remove('active');
282     });
283     this.classList.add('active');
284     // 2. Extract the URL stored on the button
285     var url = this.getAttribute('data-pdb-url');
286     // 3. Update the 3D Viewer
287     if (viewerInstance) {
288       viewerInstance.plugin.clear(); // Wipe the old structure
289       await viewerInstance.loadStructureFromUrl(url, 'pdb', false); // Load the new one
290     }
291   });
292 });

```

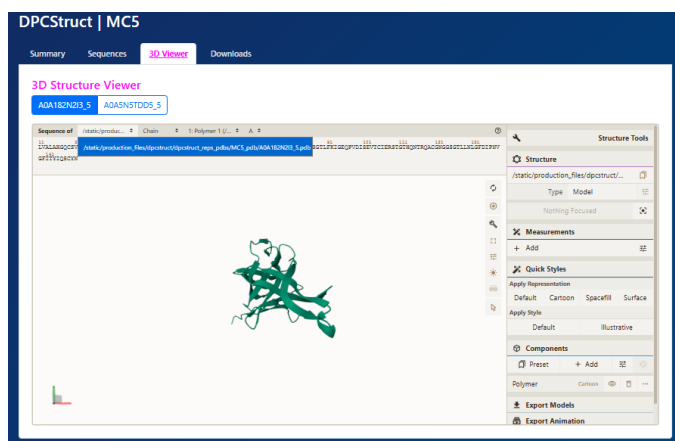
`querySelectorAll` grabs all the buttons we made. For each one, we add a `click` listener. When clicked, it deletes the current molecule with `.clear()` and repeats the loading process with the new URL.

2.4 Putting Everything together

A user requests MC5 that has two structures:

`protein_A.pdb = A0A5N5TDD5_5.pdb` and `protein_B.pdb = A0A182N2I3_5.pdb`.

1. Django passes `[{name: "protein_A.pdb", url: "/static/.../protein_A.pdb"}, {name: "protein_B.pdb", url: "/static/.../protein_B.pdb"}]` into the context.
2. The template generates two HTML buttons based on this list.
3. The `json_script` writes the array into a hidden HTML script block.
4. The user lands on the Summary tab. Mol is not loaded yet; saving memory.
5. The user clicks the 3D Viewer tab.
6. The DOM Event Listener catches the tab open event. It reads the hidden script block, finds `/static/.../protein_A.pdb` (the first item), and passes it to Mol*.
7. The WebGL canvas appears displaying `protein_A`'s 3D structure interacting in real-time.



8. The user clicks the button for `protein_B`.
9. The click event fires. It reads `/static/.../protein_B.pdb` from the clicked button's `data-pdb-url` attribute, clears the viewer, and smoothly draws the new structure.

3 Conclusion

The procedure described above allows us to explore the 3D structures of DPCstruct representative PDB(s) per metacluster in our web app. Any feedback to improve it would be much appreciated. Likewise, I am about to kick off the typing of the thesis in Overleaf; you may have received a link as viewer/editor. Before thinking about how to improve our user interface and fill out some placeholders within the web app, two issues remain:

1. Several Metaclusters in DPCFamB (e.g., MC11, MC23, MC29, ...) present some inconsistencies with the Russo et al. (2022) paper (**Pfam DA = "UNK", % DA = 100, % Average Overlap = 0.0, Overlap label = "Shifted"**). I wanted to modify them, but we need to discuss and conclude whether we can drop DPCFamB or modify it (probably requiring sudo/authors' permission).
2. When a user has chosen Pfam + Valid ID, we are displaying matching samples in both DPCFam and DPCstruct using respectively the average overlap and the average score. However, we do not have a single score for Pfam ID - DPCstruct MCID; we are using the given global score corresponding to Pfam ID - (List of 5 MCs max). I would appreciate your suggestion.

References

Russo, E. T., Barone, F., Bateman, A., Cozzini, S., Punta, M., and Laio, A. (2022). Dpcfam: unsupervised protein family classification by density peak clustering of large sequence datasets. *PLOS Computational Biology*, 18(10):e1010610.